Trajectory Planning

Trajectory Planning

1. Course Content
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

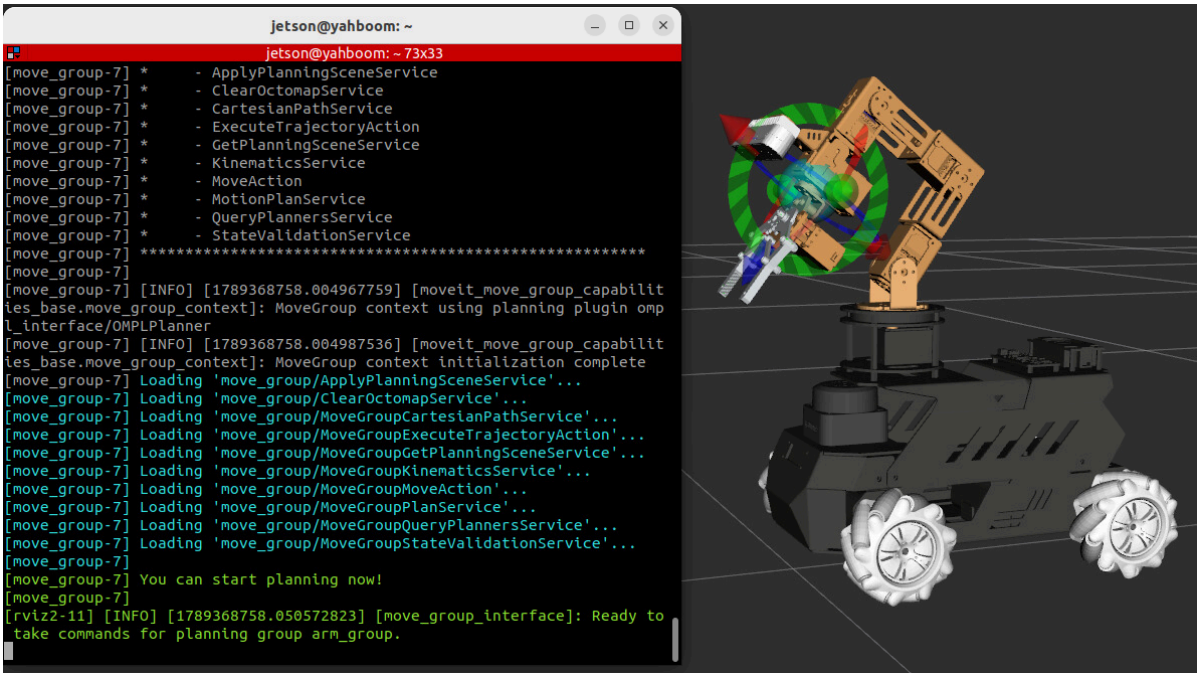
1. Course Content

This section explains how to display the path planned by MoveIt, with the robot arm moving along the planned path trajectory.

2. Preparation

```
ros2 launch yahboomcar_bringup moveit_simulation.launch.py sim2real:=true
rviz_config:=demo
```

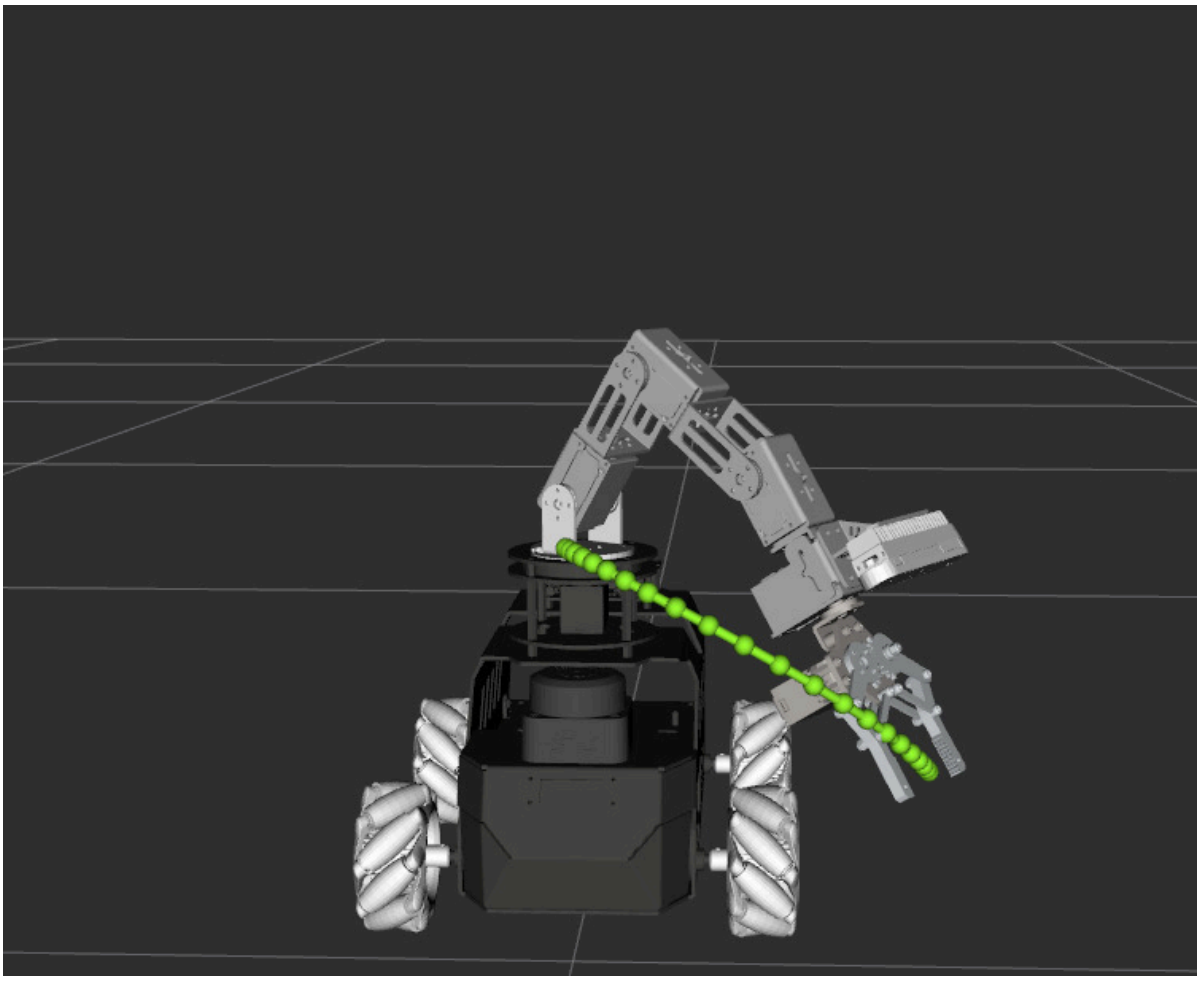
Once the program starts, when **You can start planning now!** appears in the terminal, it means the startup was successful, as shown in the figure below:

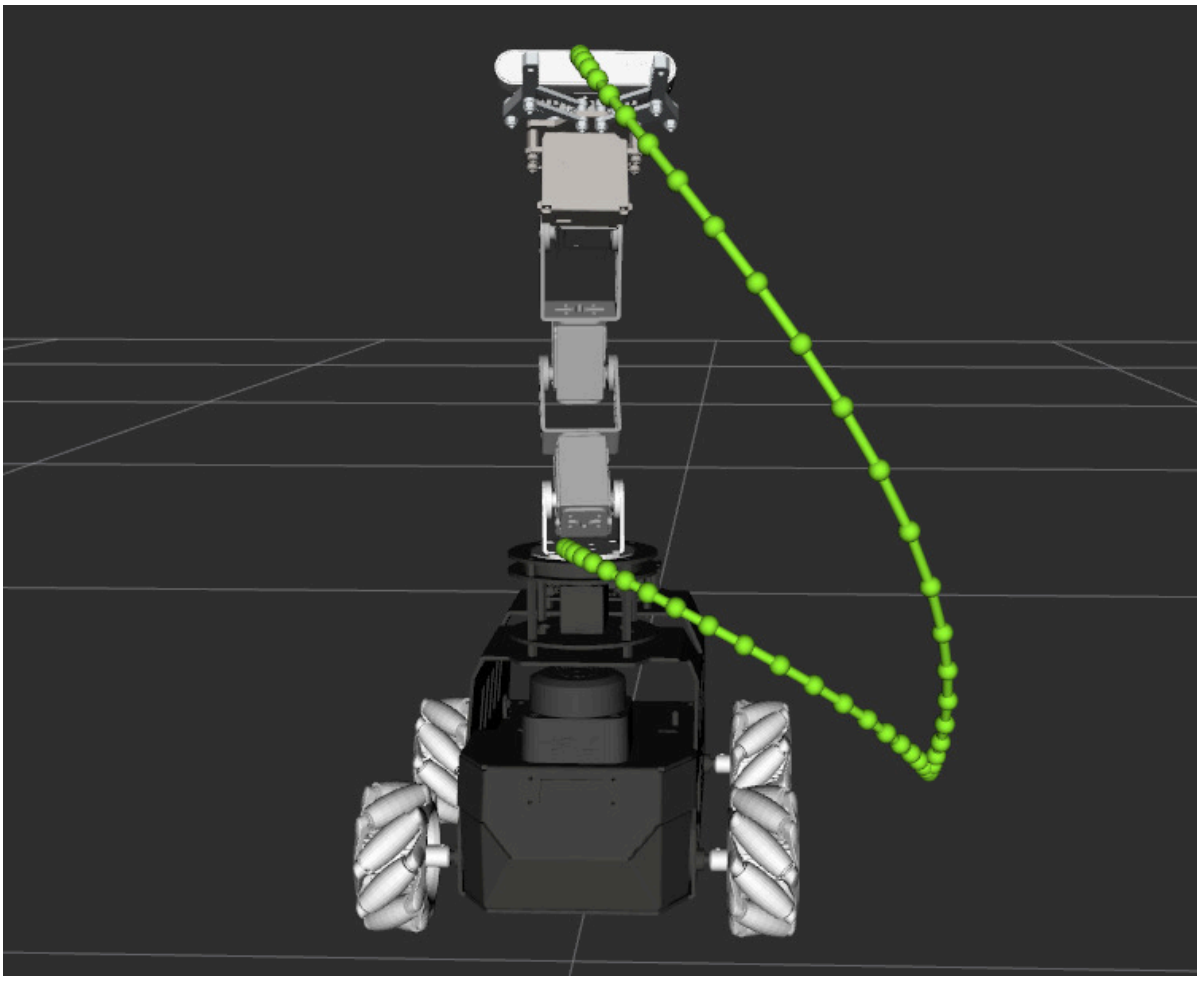


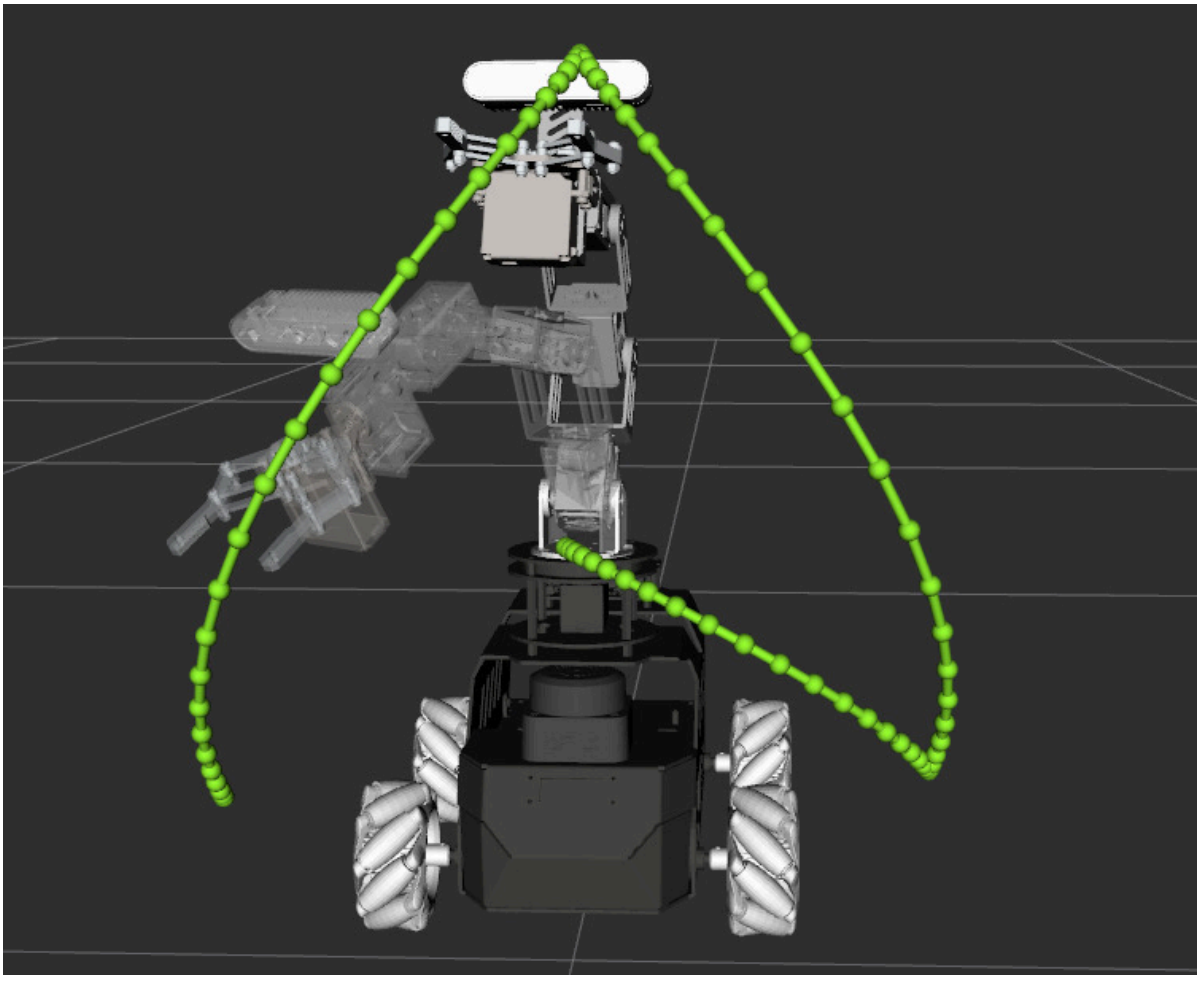
3. Case Demonstration

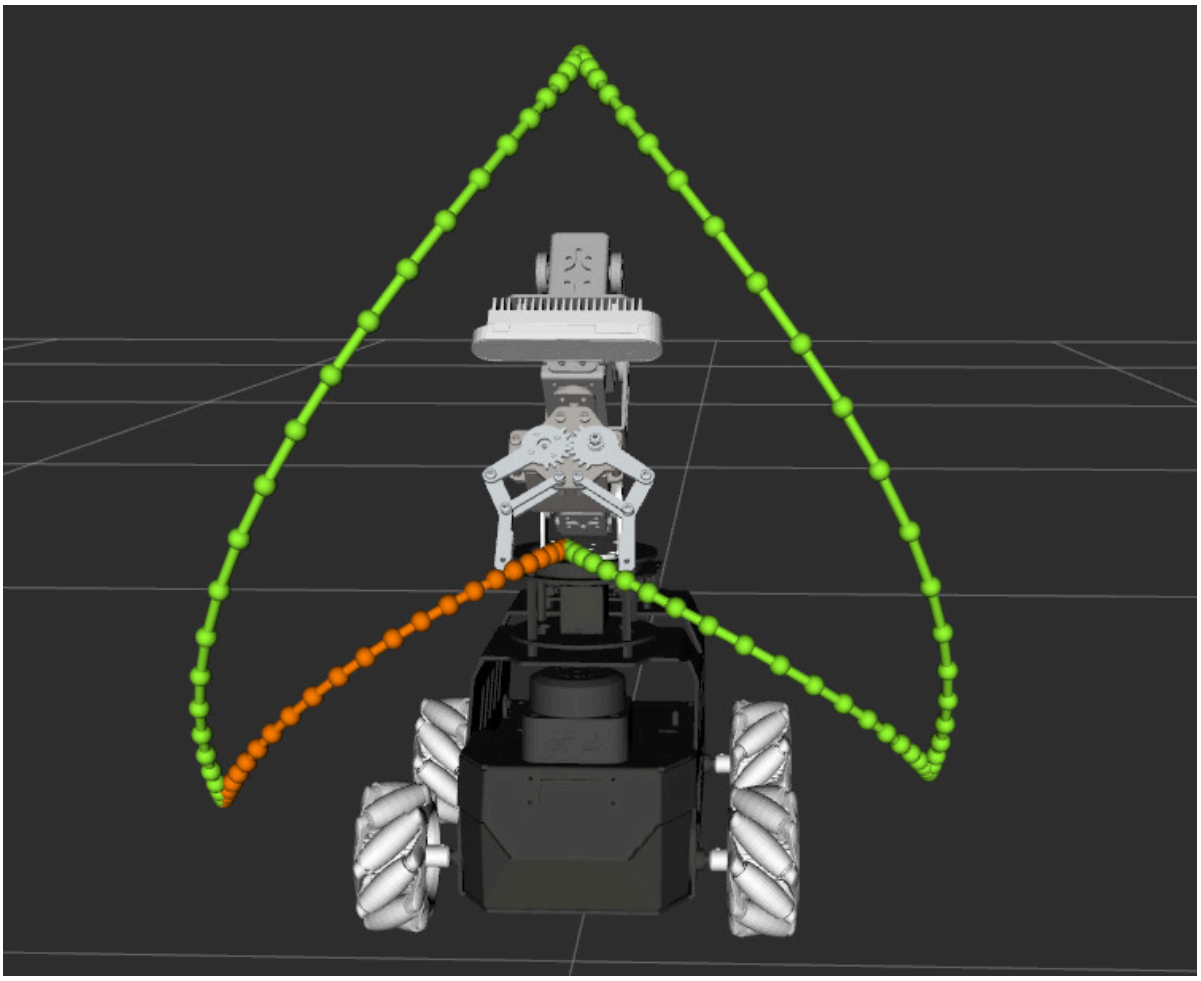
```
ros2 run moveit_demo multi_track_motion
```

After the program runs, the trajectory is displayed in RViz and the robot arm moves along the trajectory to three positions, and finally the robot arm returns to the initial position, as shown in the figure below:









4. Core Source Code Analysis

- Source code path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/moveit_demo/src/multi_track_motion.cpp
```

initialize function description:

- `MoveGroupInterface(shared_from_this(), "arm_group")`: Creates a MoveGroup control interface, bound to the `arm_group` planning group.
- `setNumPlanningAttempts(10)`: Sets the number of planning attempts per call to 10.
- `setPlanningTime(5.0)`: Sets the maximum time for a single planning attempt to 5 seconds.
- `setMaxVelocityScalingFactor(1.0)` / `setMaxAccelerationScalingFactor(1.0)`: Both velocity and acceleration scaling factors are 1.0.
- `setEndEffectorLink("gripper_tcp")`: Specifies the end effector link as `gripper_tcp`.
- `setPoseReferenceFrame("measure")`: Sets the pose reference coordinate frame to measure.
- `getJointModelGroup("arm_group")`: Gets the arm_group planning group, used to reference the joint model during visualization.
- `MoveItVisualTools(shared_from_this(), "measure", "multi_track", ...)`: Creates an RViz visualization tool, with the reference frame being measure and the published topic being multi_track.
- `deleteAllMarkers()`: Clears all existing markers in RViz.

- `loadRemoteControl()`: Loads the RViz remote control buttons.

Record initial joint angles:

- `getCurrentJointValues()`: Gets the current joint angle values (radians) of the robot arm, used to finally return to the initial position.
- `getVariableNames()`: Gets the list of joint names, printed one-to-one with the angle values.

Multi-target trajectory motion:

- `target_groups`: Defines 3 groups of target joint angles (radians), each with 5 joint values.
- `setJointValueTarget(joints)`: Sets the current target joint angles.
- `plan(plan)`: Plans a trajectory for the target joint angles.
- `publishTrajectoryLine(...)`: Draws the motion trajectory line of the end effector in cyan-green in RViz.
- `trigger()`: Triggers a render update so that the trajectory is visible in RViz.
- `execute(plan)`: If planning succeeds, executes the trajectory and drives the robot arm to the target pose.
- `points.back()`: Takes the last point of the trajectory and prints the joint angles after reaching the target.
- `continue`: If the current group fails to plan, skips it and continues with the next group.

Return to the initial position:

- `setJointValueTarget(initial_joints)`: Sets the target to the recorded initial joint angles.
- `publishTrajectoryLine(..., rviz_visual_tools::ORANGE)`: Draws the return trajectory line in orange in RViz, distinguishing it from the forward trajectory.
- `execute(plan_home)`: Executes the return trajectory, and the robot arm returns to the initial pose.
- `rclcpp::shutdown()`: Shuts down the node after the task is complete.

```
class MultiTrackMotion : public rclcpp::Node
{
public:
  MultiTrackMotion()
    : Node("multi_track_motion")
  {
  }

  void initialize()
  {
    // ===== 1. MoveGroupInterface =====
    move_group_interface_ =
      std::make_shared<moveit::planning_interface::MoveGroupInterface>(
        shared_from_this(), "arm_group");

    move_group_interface_->setNumPlanningAttempts(10);
    move_group_interface_->setPlanningTime(5.0);
    move_group_interface_->setMaxVelocityScalingFactor(1.0);
    move_group_interface_->setMaxAccelerationScalingFactor(1.0);
    move_group_interface_->setEndEffectorLink("gripper_tcp");
```

```
  move_group_interface_->setPoseReferenceFrame("measure");

  // ===== 2. Visual tools =====
  const moveit::core::JointModelGroup* joint_model_group =
    move_group_interface_->getRobotModel()->getJointModelGroup("arm_group");
  visual_tools_ = std::make_shared<moveit_visual_tools::MoveItVisualTools>(
    shared_from_this(), "measure", "multi_track",
    move_group_interface_->getRobotModel());
  visual_tools_->deleteAllMarkers();
  visual_tools_->loadRemoteControl();

  // ===== 4. Record initial joint angles =====
  const std::vector<double> initial_joints =
    move_group_interface_->getCurrentJointValues();
  {
    const auto& jmg_names = joint_model_group->getVariableNames();
    RCLCPP_INFO(this->get_logger(),
      "=== Initial joint angles recorded ===");
    for (size_t i = 0; i < initial_joints.size() && i < jmg_names.size(); i++)
    {
      RCLCPP_INFO(this->get_logger(), "  %s = %.4f rad (%.2f deg)",
        jmg_names[i].c_str(), initial_joints[i],
        initial_joints[i] * 180.0 / M_PI);
    }
  }


  const std::vector<std::vector<double>> target_groups = {
    {0.506145, 0.942478, -0.069813, 1.274090, 1.553343},
    {1.500983, 2.373648,  0.506145, 0.785398, 1.553343},
    {2.513274, 0.663225,  0.488692, 0.820305, 1.553343}
  };

  for (size_t g = 0; g < target_groups.size(); g++)
  {
    const auto& joints = target_groups[g];
    RCLCPP_INFO(this->get_logger(),
      "=== Group %zu: planning to [%.4f, %.4f, %.4f, %.4f, %.4f] rad ===",
      g + 1, joints[0], joints[1], joints[2], joints[3], joints[4]);

    move_group_interface_->setJointValueTarget(joints);

    moveit::planning_interface::MoveGroupInterface::Plan plan;
    if (move_group_interface_->plan(plan) !=
        moveit::core::MoveItErrorCode::SUCCESS)
    {
      RCLCPP_ERROR(this->get_logger(),
        "Group %zu planning failed!", g + 1);
      continue;
    }

    // Visualize trajectory in RViz
    visual_tools_->publishTrajectoryLine(plan.trajectory_,
      move_group_interface_->getRobotModel()->getLinkModel("gripper_tcp"),
      joint_model_group, rviz_visual_tools::LIME_GREEN);
    visual_tools_->trigger();
```

```
      RCLCPP_INFO(this->get_logger(), "Group %zu executing...", g + 1);
      move_group_interface_->execute(plan);

      // Print final joint angles
      const auto& points = plan.trajectory_.joint_trajectory.points;
      if (!points.empty())
      {
        const auto& names = plan.trajectory_.joint_trajectory.joint_names;
        const auto& last = points.back();
        RCLCPP_INFO(this->get_logger(), "Group %zu complete, joint angles:", g +
1);
        for (size_t i = 0; i < names.size() && i < last.positions.size(); i++)
        {
          RCLCPP_INFO(this->get_logger(), "  %s = %.4f rad (%.2f deg)",
            names[i].c_str(), last.positions[i],
            last.positions[i] * 180.0 / M_PI);
        }
      }
    }

    // ===== 6. Return to initial joint angles =====
    {
      const auto& jmg_names = joint_model_group->getVariableNames();
      RCLCPP_INFO(this->get_logger(),
        "=== Returning to initial position: [%.4f, %.4f, %.4f, %.4f, %.4f] rad
===",
        initial_joints[0], initial_joints[1], initial_joints[2],
        initial_joints[3], initial_joints[4]);

      move_group_interface_->setJointValueTarget(initial_joints);

      moveit::planning_interface::MoveGroupInterface::Plan plan_home;
      if (move_group_interface_->plan(plan_home) !=
          moveit::core::MoveItErrorCode::SUCCESS)
      {
        RCLCPP_ERROR(this->get_logger(),
          "Return to initial position planning failed!");
      }
      else
      {
        // Visualize return trajectory in RViz
        visual_tools_->publishTrajectoryLine(plan_home.trajectory_,
          move_group_interface_->getRobotModel()->getLinkModel("gripper_tcp"),
          joint_model_group, rviz_visual_tools::ORANGE);
        visual_tools_->trigger();

        RCLCPP_INFO(this->get_logger(),
          "Executing return to initial position...");
        move_group_interface_->execute(plan_home);

        // Print actual final joint angles after return
        const auto& points = plan_home.trajectory_.joint_trajectory.points;
        if (!points.empty())
        {
          const auto& names = plan_home.trajectory_.joint_trajectory.joint_names;
```

```
          const auto& last = points.back();
          RCLCPP_INFO(this->get_logger(),
            "Return to initial position complete, joint angles:");
          for (size_t i = 0; i < names.size() && i < last.positions.size(); i++)
          {
            RCLCPP_INFO(this->get_logger(), "  %s = %.4f rad (%.2f deg)",
              names[i].c_str(), last.positions[i],
              last.positions[i] * 180.0 / M_PI);
          }
        }
      }
    }

    rclcpp::shutdown();
  }

private:
  std::shared_ptr<moveit::planning_interface::MoveGroupInterface>
move_group_interface_;
  std::shared_ptr<moveit_visual_tools::MoveItVisualTools> visual_tools_;
};
```